

Lista Avaliativa 1 - Respostas

1)

Máquina Estendida: O sistema operacional atua como uma camada de abstração sobre o hardware, ocultando sua complexidade e fornecendo uma interface mais simples e amigável. Ele transforma o hardware "bruto" em uma máquina virtual mais fácil de programar, oferecendo chamadas de sistema padronizadas para operações como leitura/escrita de arquivos, criação de processos e comunicação de rede, sem que o programador precise conhecer detalhes de baixo nível do hardware.

Gerenciador de Recursos: O SO gerencia e coordena o uso dos recursos de hardware (CPU, memória, dispositivos de E/S, arquivos) entre múltiplos programas e usuários. Ele decide quem usa qual recurso, quando, por quanto tempo e resolve conflitos quando há competição por recursos. O objetivo é garantir uso eficiente, justo e seguro dos recursos do sistema.

2)

O processador opera em dois modos distintos para garantir a segurança e estabilidade do sistema. O modo usuário é um modo restrito onde os processos das aplicações executam com acesso limitado aos recursos do sistema. Neste modo, os programas não podem executar instruções privilegiadas nem acessar diretamente o hardware, o que impede que aplicações maliciosas ou com erros comprometam todo o sistema.

Já o modo supervisor, também chamado de modo kernel, é um modo privilegiado que possui acesso total ao hardware e pode executar qualquer instrução da CPU, além de acessar qualquer região da memória. É neste modo que o sistema operacional executa suas operações críticas, gerenciando recursos e coordenando o funcionamento de todo o computador.

A troca entre esses modos acontece em momentos específicos durante a operação do sistema. Quando um programa em modo usuário precisa realizar uma operação que requer privilégios, como acessar um arquivo ou alocar memória, ele faz uma chamada de sistema que transfere a execução para o modo supervisor. Além disso, quando ocorrem interrupções de hardware, como o acionamento de um dispositivo de entrada e saída, ou exceções causadas por erros no programa o processador alterna automaticamente para o modo supervisor para tratar essas situações. Após concluir o tratamento da chamada de sistema ou da interrupção, o processador retorna ao modo usuário, permitindo que o programa continue sua execução normal.

3)

O sistema de proteção de arquivos no UNIX utiliza um código de 9 bits para definir as permissões de acesso de cada arquivo. Esses 9 bits são divididos em três grupos de 3 bits, que representam as permissões do proprietário do arquivo, do grupo ao qual ele pertence e de outros usuários.

Cada grupo de 3 bits indica se há permissão de leitura (r), escrita (w) e execução (x). Por exemplo, o código `rwxr-x--x` significa que o proprietário pode ler, escrever e executar o arquivo; os usuários do mesmo grupo podem ler e executar, mas não escrever; e os demais usuários podem apenas executar. Assim, esse sistema permite controlar de forma simples e eficiente quem pode acessar e modificar os arquivos no sistema.

4)

Um processo é a abstração de um programa em execução, representando uma instância ativa que possui seus próprios recursos, como código, dados e estado de execução. Quando um processo não está em execução (ou seja, está suspenso ou bloqueado), suas informações são armazenadas em uma estrutura de dados mantida pelo sistema operacional chamada tabela de processos. Cada entrada nesta tabela armazena todo o contexto necessário para que o processo possa ser retomado posteriormente, incluindo o valor dos registradores da CPU (como o contador de programa), seu estado atual (pronto, bloqueado), prioridade e informações sobre os recursos que ele utiliza, como arquivos abertos e memória alocada.

5)

O espaço de endereçamento é o conjunto de endereços de memória lógicos que um processo pode referenciar. Ele define uma visão privada e isolada da memória para cada processo, geralmente iniciando no endereço 0 e indo até um valor máximo. Este espaço contém o código do programa, seus dados (variáveis globais) e a pilha de execução (variáveis locais e chamadas de função), garantindo que um processo não possa acessar diretamente a memória de outro.

A memória virtual é um mecanismo que desacopla o espaço de endereçamento lógico de um processo da memória física (RAM) do computador. Essa técnica permite que o sistema operacional mantenha apenas as partes ativamente utilizadas de um processo na RAM, enquanto o restante é armazenado em disco. Isso cria a ilusão de que o sistema possui muito mais memória do que a fisicamente disponível, possibilitando a execução de programas maiores que a RAM e aumentando o grau de multiprogramação do sistema.

6)

A estrutura de micronúcleo organiza o sistema operacional de forma a minimizar a quantidade de código que executa em modo supervisor (núcleo). A ideia principal é mover a maior parte dos serviços tradicionalmente encontrados no núcleo, como sistemas de arquivos, drivers de dispositivos e gerenciadores de rede, para o espaço do usuário, onde eles executam como processos servidores.

Dessa forma, o núcleo se torna mínimo, sendo responsável apenas pelas funções mais essenciais, como a comunicação entre processos (IPC), o escalonamento básico de threads e o gerenciamento de baixo nível da memória. Essa abordagem aumenta a robustez e a segurança do sistema, pois uma falha em um serviço (como um driver) executando em modo usuário não compromete todo o sistema operacional, como ocorreria em uma arquitetura monolítica.

7)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>

#define NUM_FILHOS 5
```

```

typedef struct {
    pid_t pid;
    int numero;
} DadosFilho;

int main() {
    int fd[2];
    pid_t pids[NUM_FILHOS];

    if (pipe(fd) == -1) {
        perror("pipe falhou");
        exit(1);
    }

    for (int i = 0; i < NUM_FILHOS; i++) {
        pids[i] = fork();

        if (pids[i] < 0) {
            perror("fork falhou");
            exit(1);
        }

        if (pids[i] == 0) {
            close(fd[0]);

            srand(time(NULL) ^ getpid());

            int numero_sorteado = rand() % 201;

            DadosFilho dados;
            dados.pid = getpid();
            dados.numero = numero_sorteado;

            printf("Filho (PID %d) enviou o número %d.\n", dados.pid,
dados.numero);

            write(fd[1], &dados, sizeof(DadosFilho));

            close(fd[1]);

            exit(0);
        }
    }

    close(fd[1]);

    int menor_numero = 201;
    pid_t pid_filho_menor = -1;

    printf("\nPai (PID %d) esperando para receber os números...\n",
getpid());

```

```

    for (int i = 0; i < NUM_FILHOS; i++) {
        DadosFilho dados_recebidos;

        read(fd[0], &dados_recebidos, sizeof(DadosFilho));

        printf("Pai recebeu %d do filho (PID %d).\n",
            dados_recebidos.numero, dados_recebidos.pid);

        if (dados_recebidos.numero < menor_numero) {
            menor_numero = dados_recebidos.numero;
            pid_filho_menor = dados_recebidos.pid;
        }
    }

    close(fd[0]);

    for (int i = 0; i < NUM_FILHOS; i++) {
        wait(NULL);
    }

    printf("\n--- Resultado Final ---\n");
    printf("O menor número recebido foi: %d\n", menor_numero);
    printf("Enviado pelo filho com PID: %d\n", pid_filho_menor);

    return 0;
}

```

8)

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <signal.h>

void filho_executa() {
    int contador = 0;

    while (1)
    {
        printf("Contador: %d\n", contador);
        contador++;
        sleep(1);
    }
}

int main() {
    pid_t filhos[3];
    for (int i = 0; i < 3; i++) {
        filhos[i] = fork();
    }
}

```

```

        if (filhos[i] == 0) {
            printf("Filho %d criado com sucesso!. Meu PID: %d\n", i,
getpid());
            filho_executa();
            exit(0);
        } else if (filhos[i] < 0) {
            perror("Erro ao criar o filho!\n");
            exit(1);
        }
    }
}

printf("Filhos criados com sucesso.\n");
sleep(1);

for (int i = 0; i < 3; i++) {
    kill(filhos[i], SIGSTOP);
}

printf("Filhos foram parados.\n");

for (int i = 0; i < 3; i++) {
    printf("Filho %d:\n", filhos[i]);
    kill(filhos[i], SIGCONT);
    sleep(10);
    kill(filhos[i], SIGSTOP);
}

for (int i = 0; i < 3; i++) {
    kill(filhos[i], SIGKILL);
}

return 0;
}

```

9)

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>

int main()
{
    pid_t filhos[10];
    srand(time(NULL));

    for (int i = 0; i < 10; i++)
    {

```

```
int numero_aleatorio = rand() % 10;
char buffer[12];
snprintf(buffer, sizeof(buffer), "%d\n", numero_aleatorio);
setenv("num", buffer, 1);
filhos[i] = fork();

if (filhos[i] == 0)
{
    char *mensagem = getenv("num");
    printf("Sou o filho com PID: %d. Minha variável de ambiente tem  
o seguinte número: %s", getpid(), mensagem);
    exit(0);
}
else if (filhos[i] < 0)
{
    perror("Erro ao criar os filhos!");
    exit(1);
}
}

for (int i = 0; i < 10; i++)
{
    wait(NULL);
}

return 0;
}
```